Introduction to Deep Learning

Lecturer: Dr. Samuel Kakuba

February 09, 2026

Index

01 Perceptron

02 Artificial Neural Network

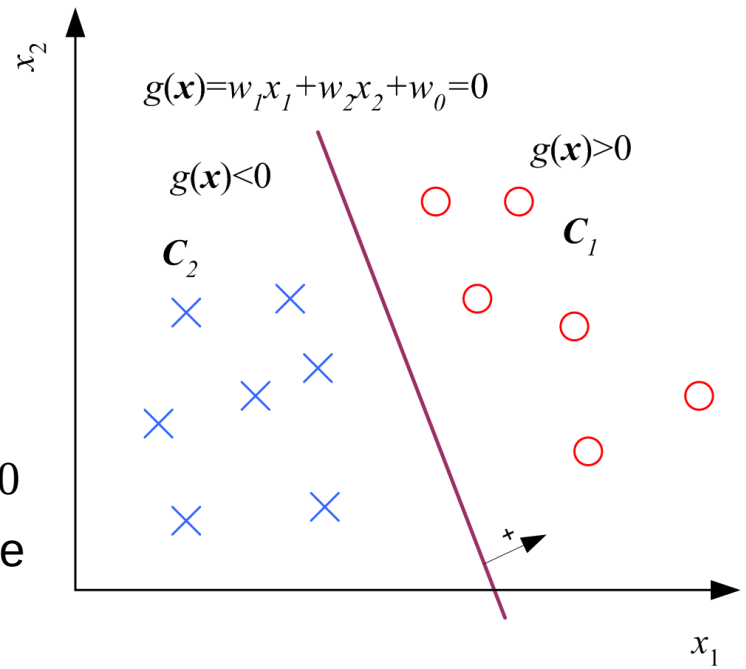
03 DL from Scratch

1

Machine Learning: Classification

- Assume that we collected a dataset with features and label
  - $\boldsymbol{f(x) = y}$
  - x: features, y: label (0 or 1, benign or malignancy)
  - What kind of $f(\cdot)$ is good enough for our task?

- Simple linear model
  - $g(x)$

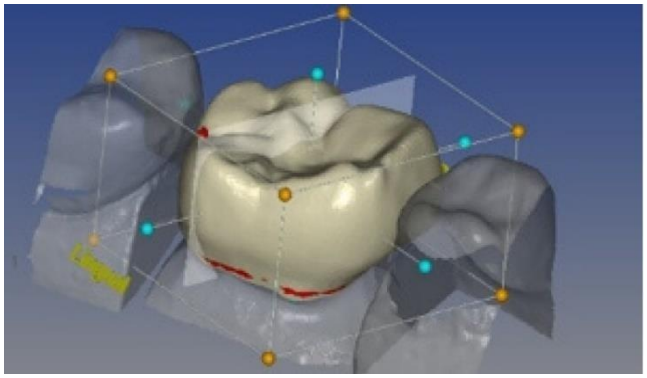


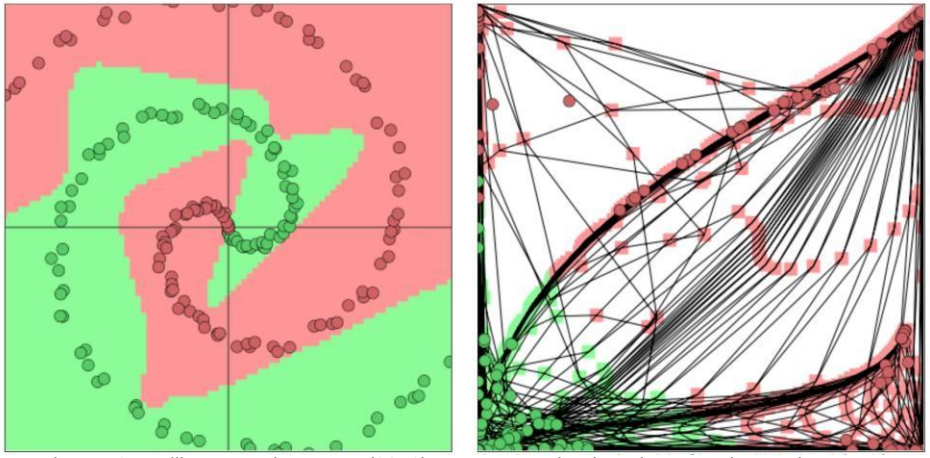
$$\text{choose} \begin{cases} C_1 & \text{if } g(\mathbf{x}) > 0 \\ C_2 & \text{otherwise} \end{cases}$$

1

Linear Model is not Enough

- Mapping high-dimensional data to output
  - **Require non-linear models** which can learn meaningful features from medical images (input)
- **Deep learning**
  - Combination of non-linear mapping → high-level abstraction
  - When rules or feature definitions are not clearly specified, complex, and difficult to discern, we can use deep learning.

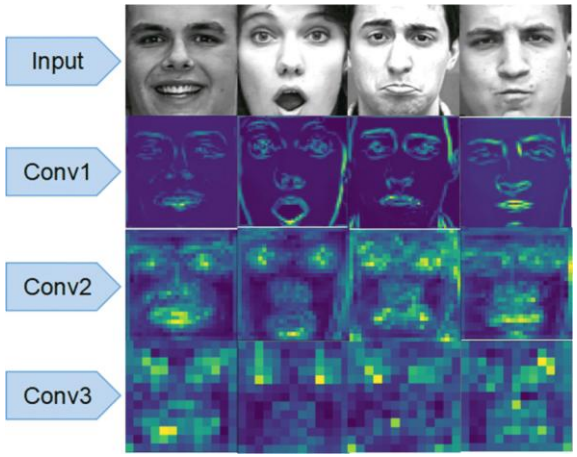


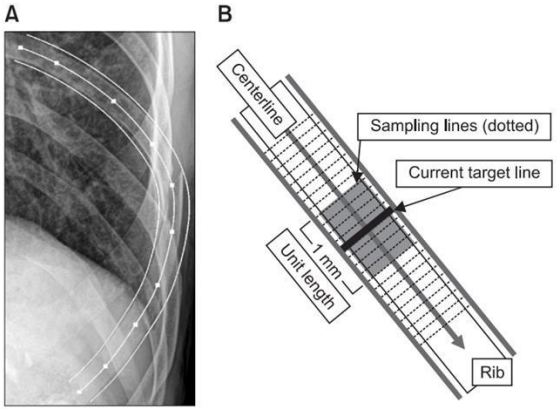


Bengio, Y., Courville, A., & Vincent, P. (2013). Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, *35*(8), 1798-1828.
https://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html

1 Feature Engineering → Feature Learning

- Feature engineering
  - Edge, texture, color, …: Hand-crafted features
- Now, data-driven approach
  - Machine determines (learns) best (or better) features for tasks
- Hierarchical learning
  - Low → high-level features
- Making a program → get a program from data



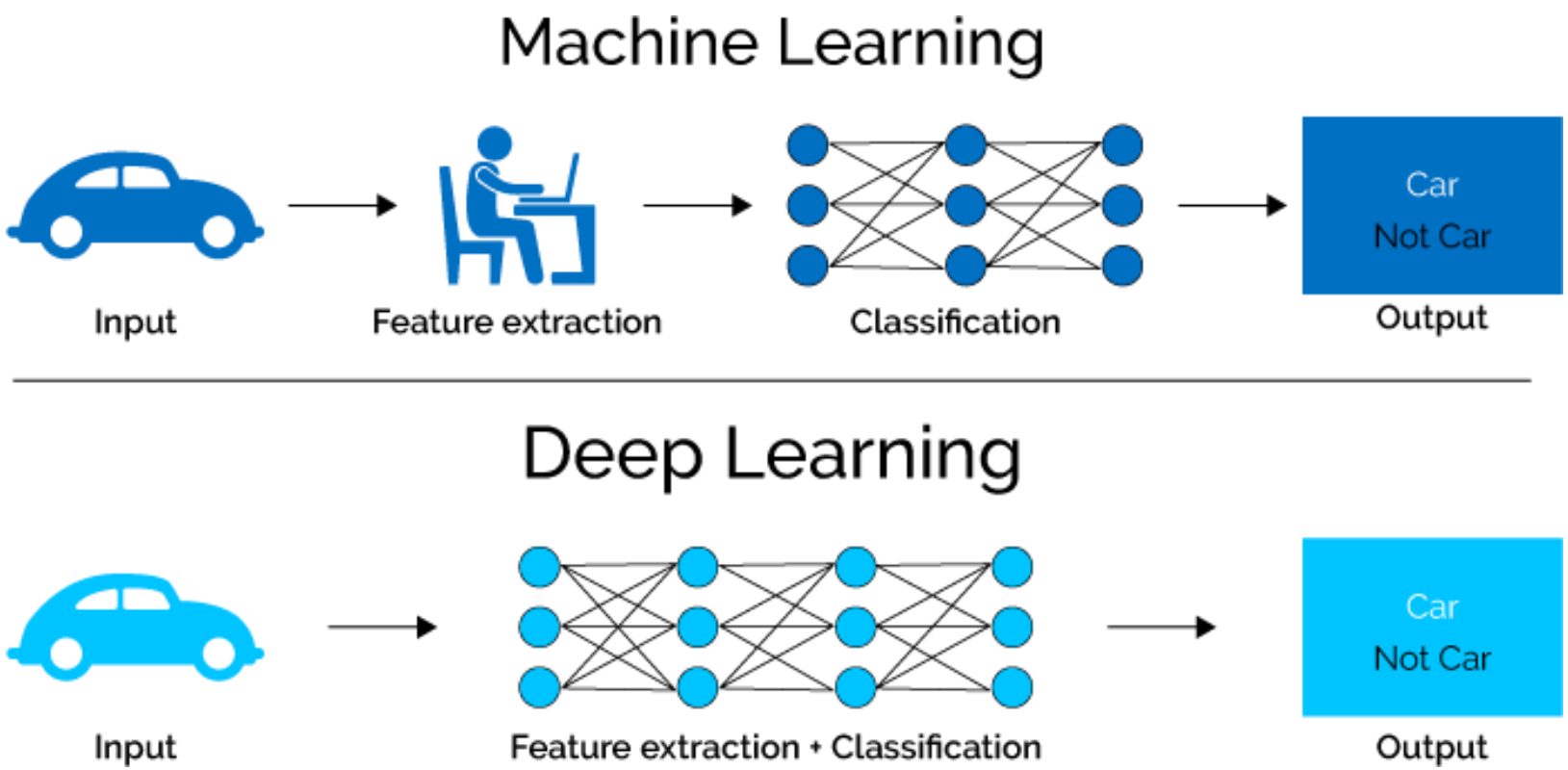


Rib ROI measurement and texture analysis
(Kim, et al. 2013)

Lindsey, Robert, et al. "Deep neural network improves fracture detection by clinicians." PNAS 115.45 (2018):

1

Linear Model is not Enough

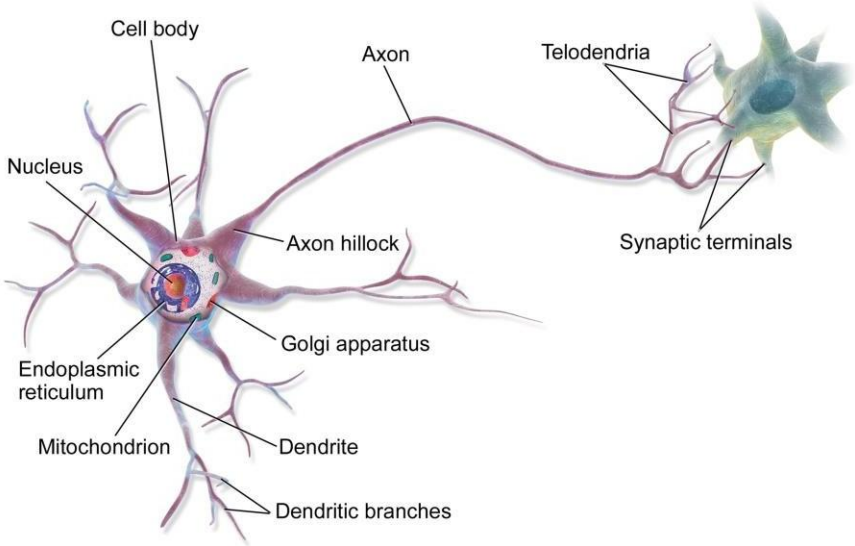


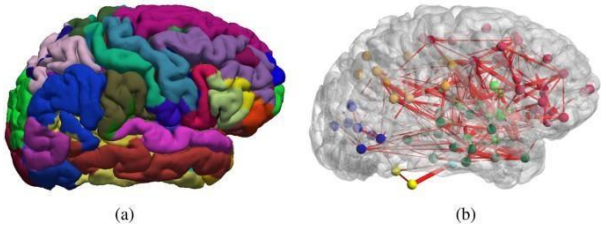
Source: Toward Data Science

1

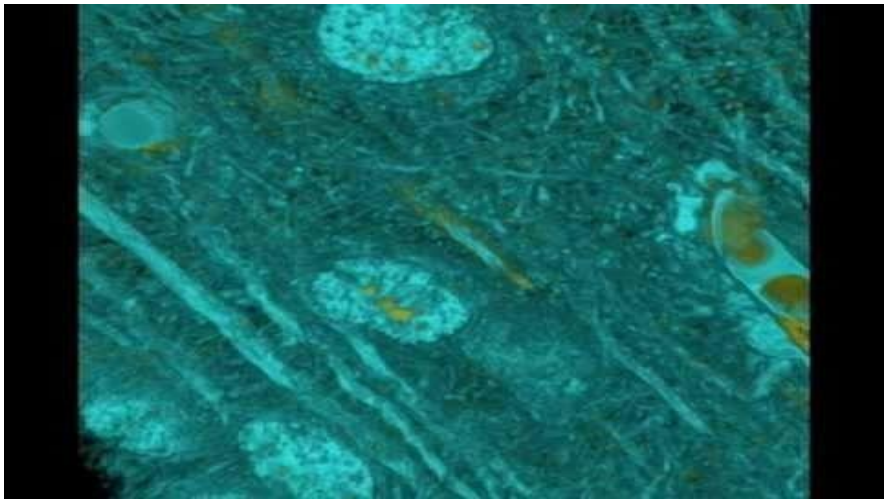
Neural Circuit

- **Neuron:** single cell which conducts an electric signal
- **Neural circuit:** a population of neurons interconnected by synapses





Large-scale brain networks



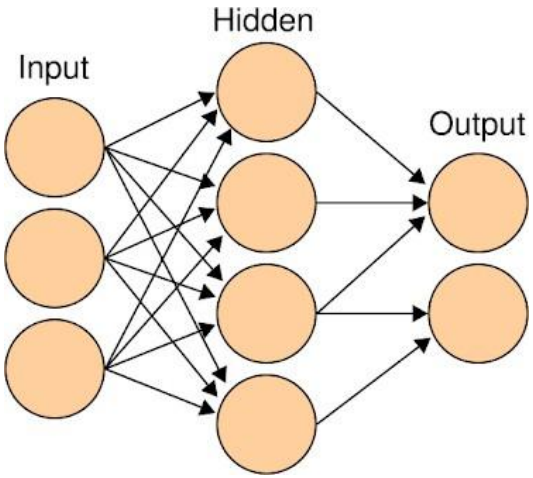
1

Artificial Neural Network

- Neural networks trained by data using trial and error
  - This is what brain does

https://developer.nvidia.com/blog/deep-learning-nutshell-reinforcement-learning/

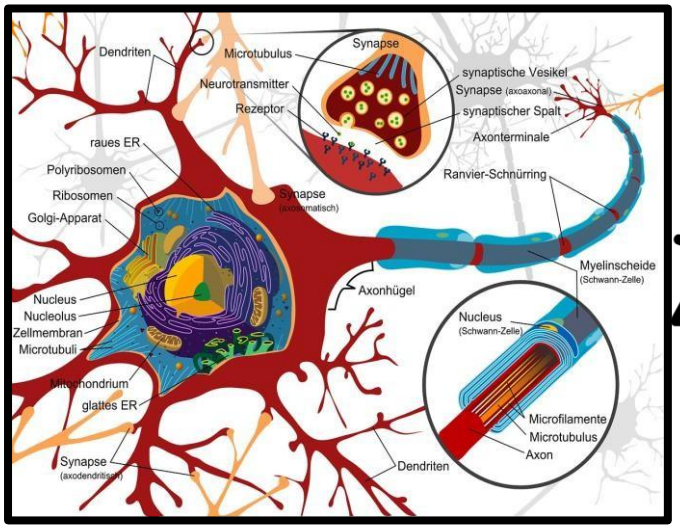
- Components
  - Nodes with activation function
    - ✓ Inspired by neurons
  - Layers: input, hidden, output

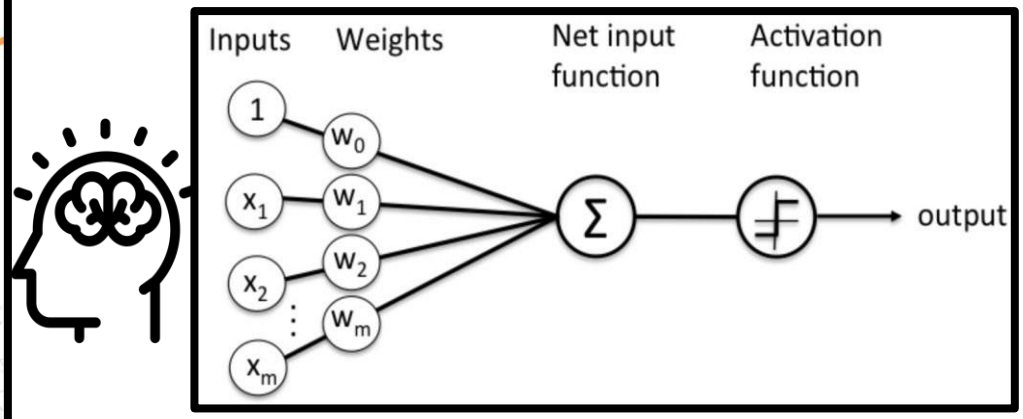


1

Perceptron

- **Perceptron (Rosenblatt, 1958)**
  → Basic unit of a neural network
  - A feedforward neural network
  - A linear classifier with an **activation function (non-linear mapping)**
  - Input * weights → activation function → output



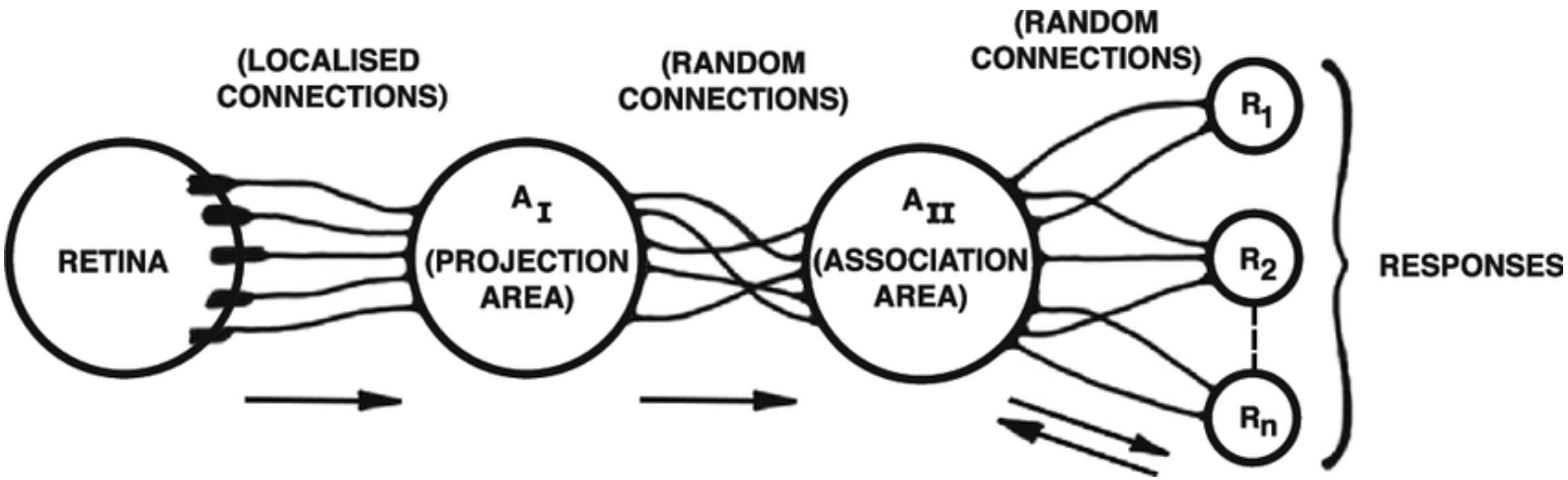


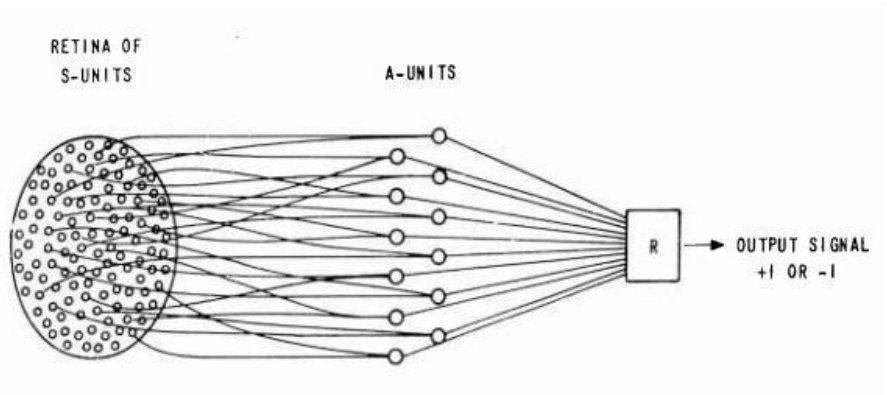
Rosenblatt's Perceptron

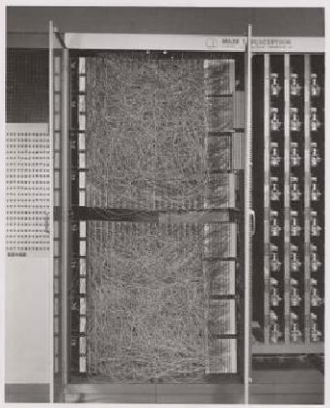
Rosenblatt, F. (1958). The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, *65*(6), 386.

1

Perceptron





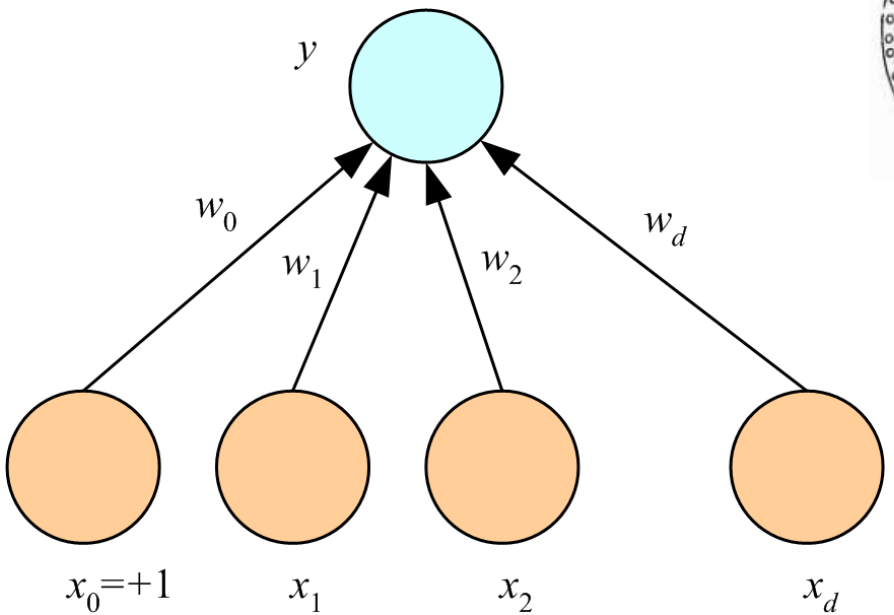


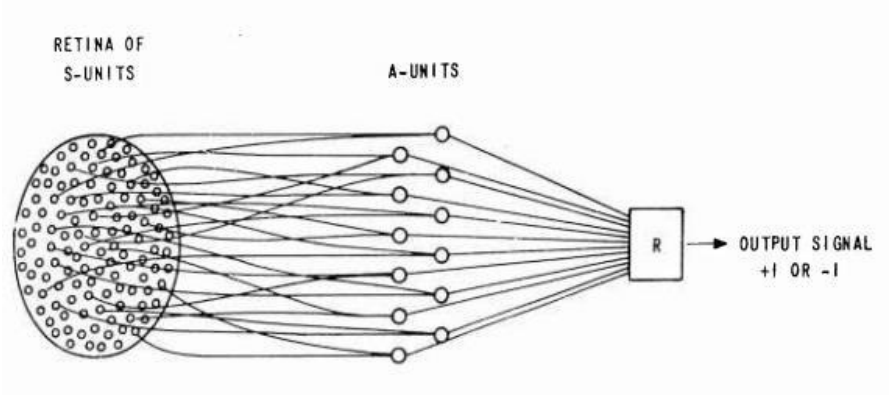
Orbach, J. "Principles of neurodynamics. Perceptrons and the theory of brain mechanisms." *Archives of General Psychiatry* 7.3 (1962): 218-219.

1

Perceptron

- **Perceptron (1962)**





$$y = \sum_{j=1}^{d} w_j x_j + w_0 = \mathbf{w}^T \mathbf{x}$$

$$\mathbf{w} = [w_0, w_1, \ldots, w_d]^T$$

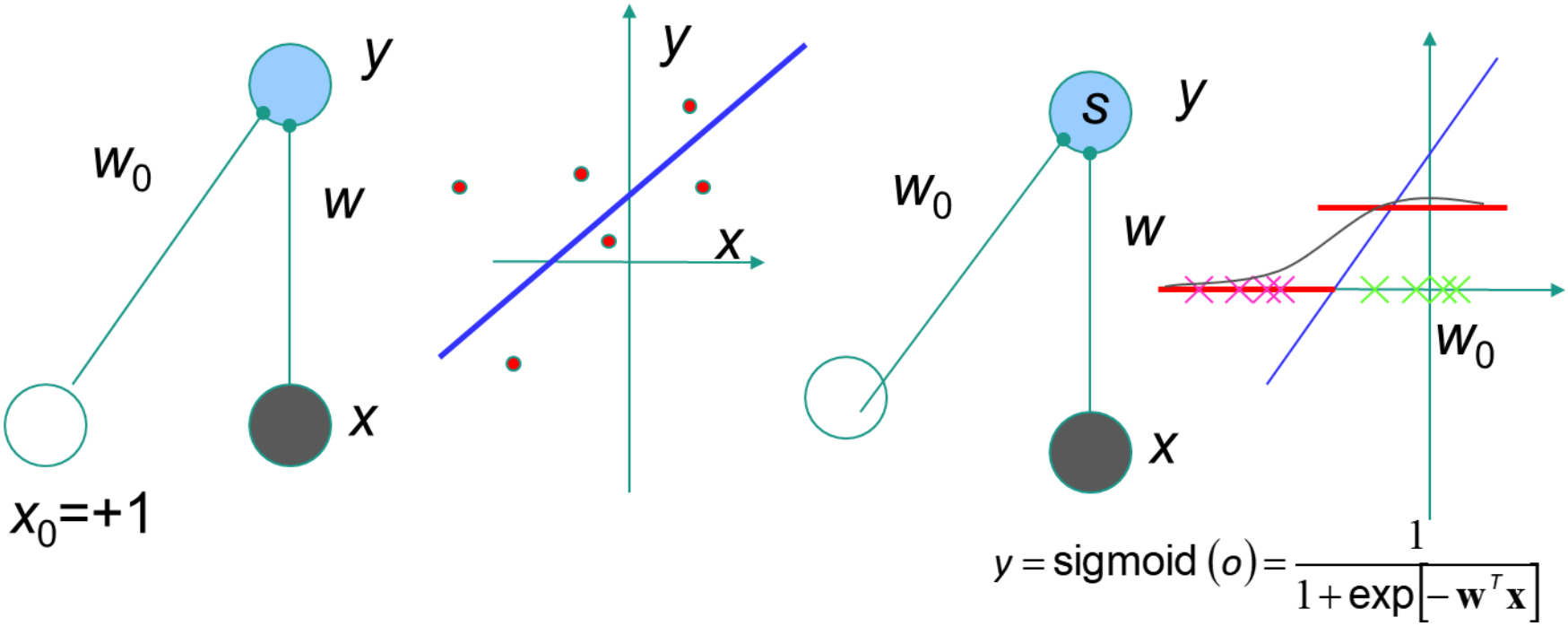
$$\mathbf{x} = [1, x_1, \ldots, x_d]^T$$

Orbach, J. "Principles of neurodynamics. Perceptrons and the theory of brain mechanisms." *Archives of General Psychiatry* 7.3 (1962): 218-219.

1

Perceptron

- Regression: $y=wx+w_0$
- Classification: $y=1(wx+w_0>0)$

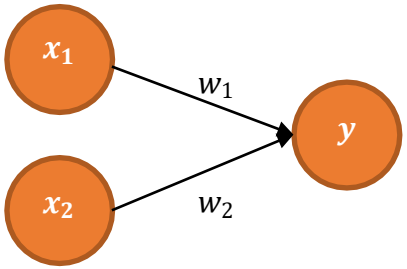


1 Perceptron

- Implementation of AND gate
  - $\Phi = \begin{cases} 0 & (w_1x_1 + w_2x_2 \le \theta) \\ 1 & (w_1x_1 + w_2x_2 > \theta) \end{cases}$
- Tutorial_2_1_Introduction.ipynb

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

AND gate



$$y_i = \sum_{j=1}^{d} w_{ij}x_j + w_{i0} = \mathbf{w}_i^T\mathbf{x}$$

$$\mathbf{y} = \mathbf{W}\mathbf{x}$$

```python
def AND(x1, x2):
    w1, w2, theta = 0.5, 0.5, 0.7
    out = x1*w1 + x2*w2
    if out <= theta:
        return 0
    else:
        return 1

print(AND(x1=0, x2=0))
print(AND(x1=1, x2=0))
print(AND(x1=0, x2=1))
print(AND(x1=1, x2=1))
```

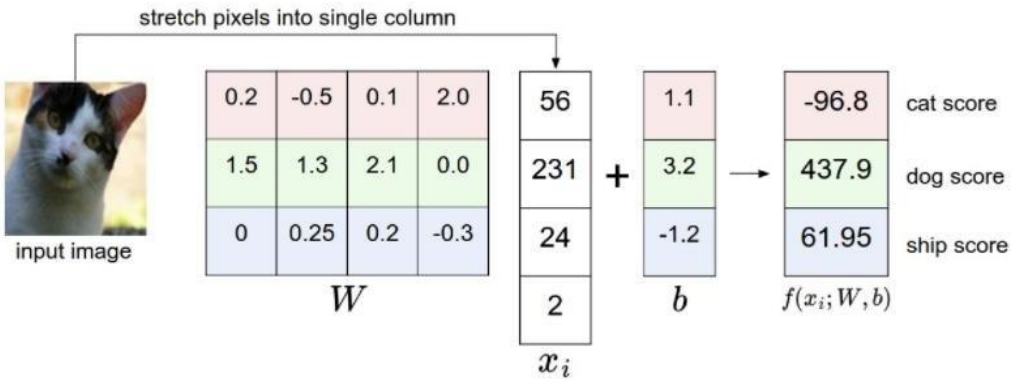
1

Perceptron

- Implementation of AND gate
  - Introducing bias
  - $\Phi \begin{cases} 0 & (b + w_1x_1 + w_2x_2 \leq 0) \\ 1 & (b + w_1x_1 + w_2x_2 > 0) \end{cases}$

```python
import numpy as np

def AND_Bias(x1, x2):
    x = np.array([x1, x2])
    w = np.array([0.5, 0.5])
    b = 0.0
    out = np.sum(w*x) + b
    if out <= 0:
        return 0
    else:
        return 1

print(AND_Bias(x1=0, x2=0))
print(AND_Bias(x1=1, x2=0))
print(AND_Bias(x1=0, x2=1))
print(AND_Bias(x1=1, x2=1))
```

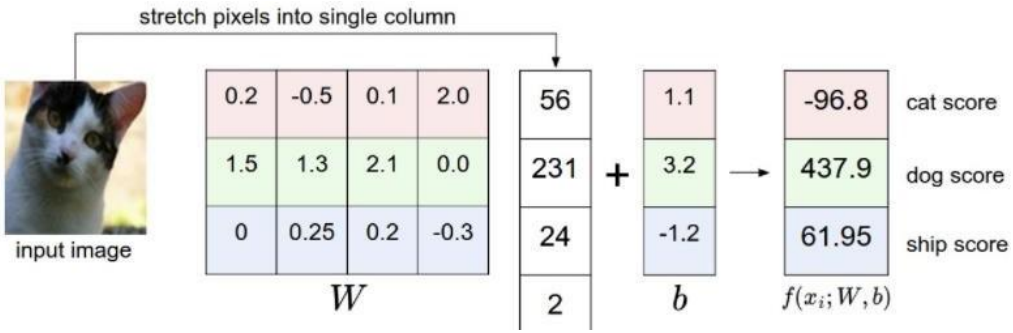
- **bias:** influences output, but not interact with the input (x)
- **parameters: weight and bias**



https://cs231n.github.io/linear-classify/

1

Perceptron



🐱 Cat score

$$
\begin{aligned}
s_{\text{cat}} &= (0.2 \times 56) + (-0.5 \times 231) + (0.1 \times 24) + (2.0 \times 2) + 1.1 \\
&= 11.2 - 115.5 + 2.4 + 4 + 1.1 \\
&= \boxed{-96.8}
\end{aligned}
$$

🐶 Dog score

$$
\begin{aligned}
s_{\text{dog}} &= (1.5 \times 56) + (1.3 \times 231) + (2.1 \times 24) + (0.0 \times 2) + 3.2 \\
&= 84 + 300.3 + 50.4 + 0 + 3.2 \\
&= \boxed{437.9}
\end{aligned}
$$

🚢 Ship score

$$
\begin{aligned}
s_{\text{ship}} &= (0 \times 56) + (0.25 \times 231) + (0.2 \times 24) + (-0.3 \times 2) - 1.2 \\
&= 0 + 57.75 + 4.8 - 0.6 - 1.2 \\
&= \boxed{61.95}
\end{aligned}
$$

https://cs231n.github.io/linear-classify/

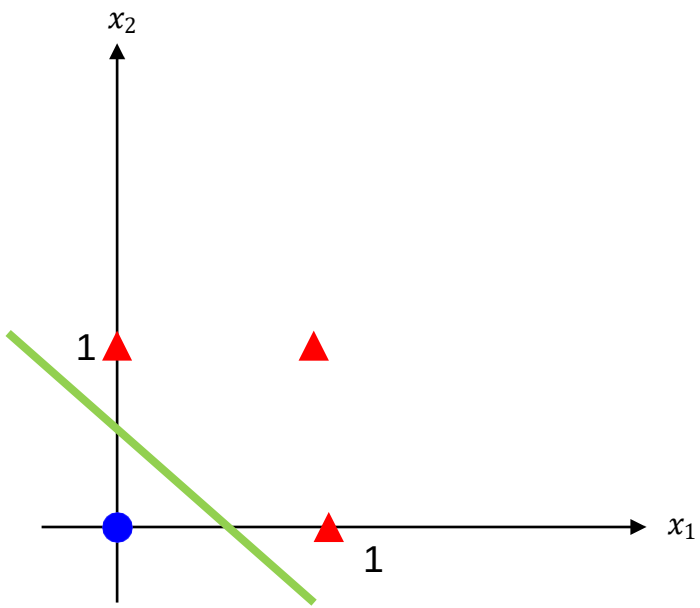
1

Perceptron

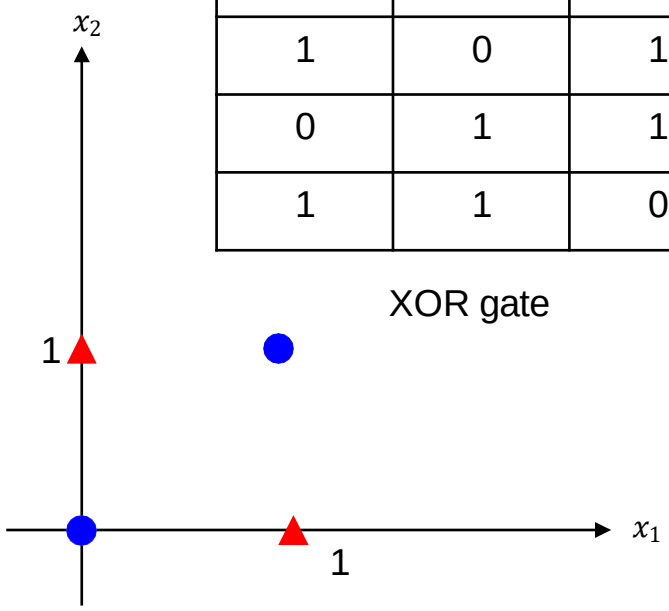
- XOR gate
  - OR gate implementation: similar to AND gate
  - How to implement XOR gate using Perceptron
  - Not possible with single perceptron

x_1	x_2	O
0	0	0
1	0	1
0	1	1
1	1	0

XOR gate



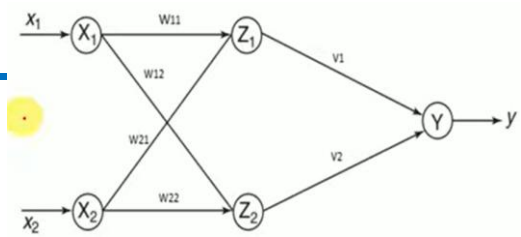
OR gate



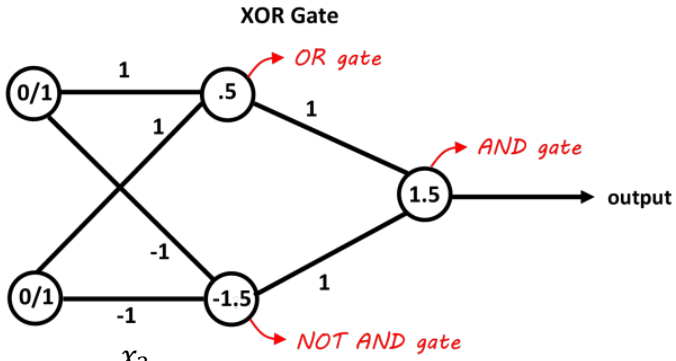
XOR gate

1

Perceptron

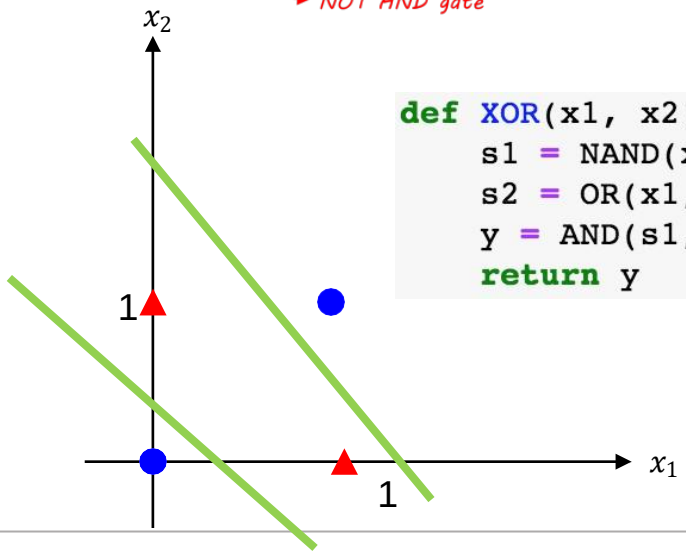


- XOR gate
  - A well-known problem with nonlinear decision boundary
  - Multi-layer perceptron



x_1	x_2	s_1	s_2	O
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0

XOR gate

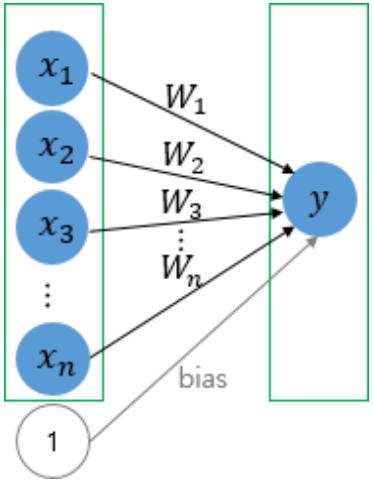



```python
def XOR(x1, x2):
    s1 = NAND(x1, x2)
    s2 = OR(x1, x2)
    y = AND(s1, s2)
    return y
```

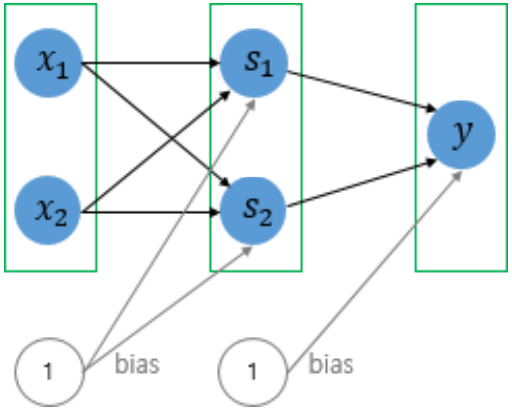
2

Artificial Neural Network

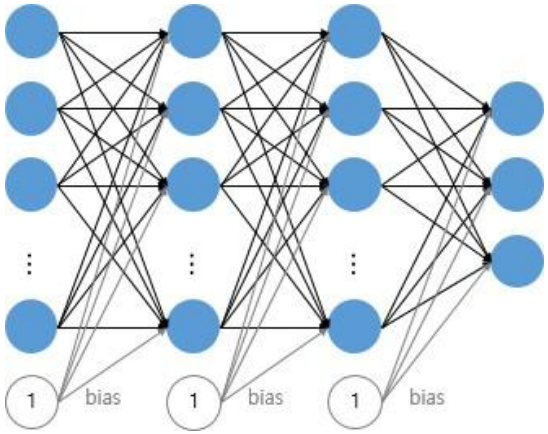
- Artificial Neural Network
  - Neural networks trained by data using trial and error
  - Input – Hidden – Output layer



Perceptron



Multi-layer Perceptron
(Artificial Neural Network)

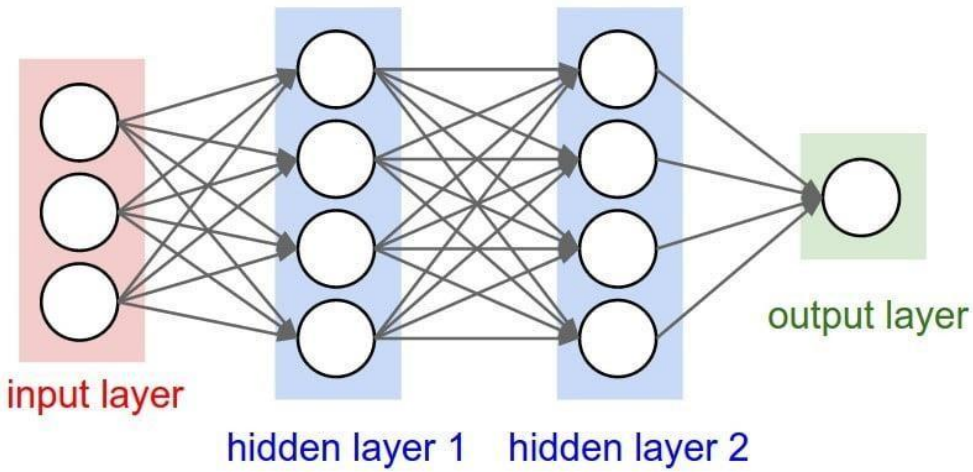


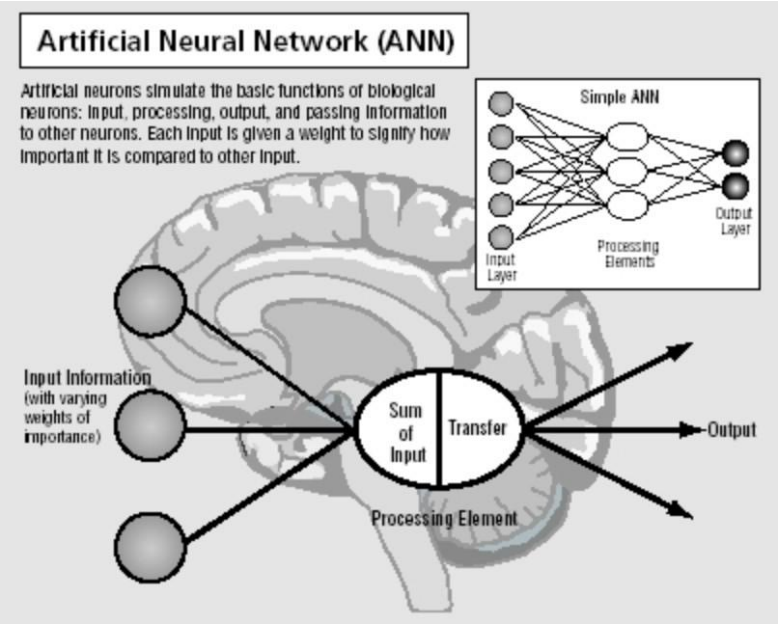
Deep Neural Network

2

Artificial Neural Network

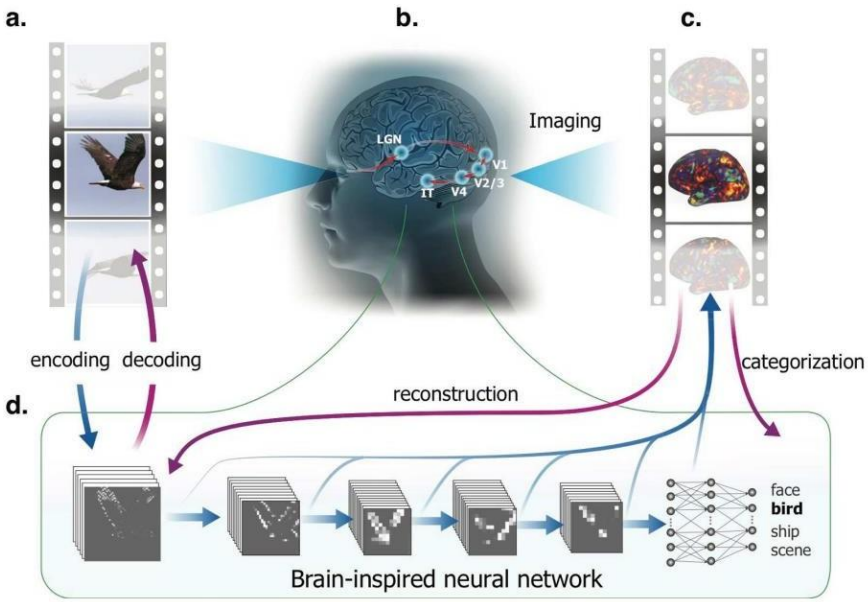
- Feedforward artificial neural network
- ANN wherein connections between nodes do not form a cycle
  - If there is cycle, it is recurrent neural network (RNN)
- Consists of at least three layers of nodes with nonlinear activation functions



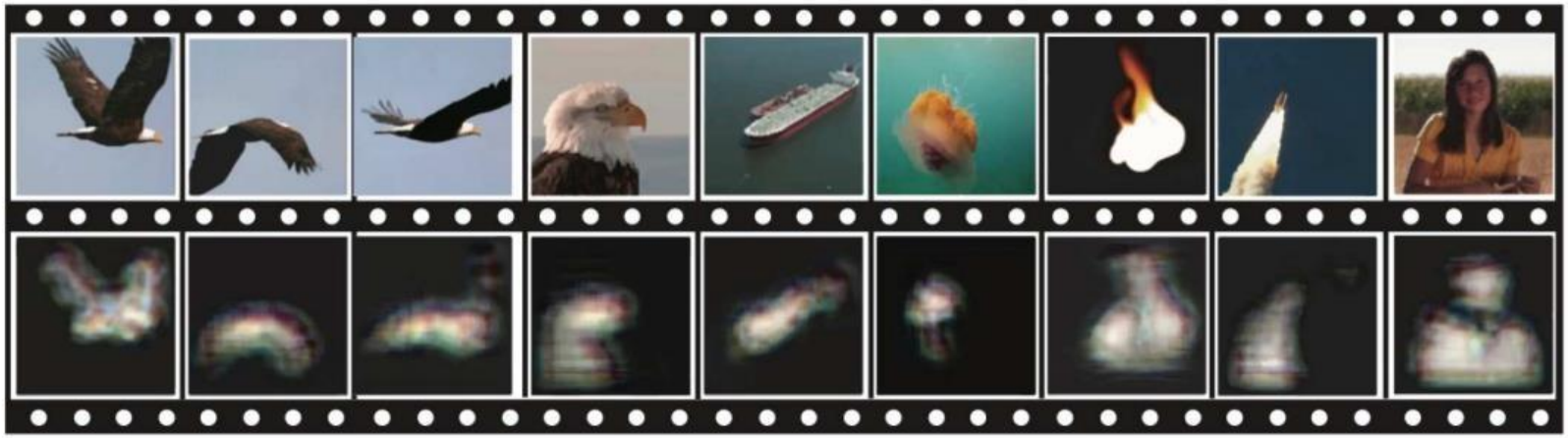


2

Artificial Neural Network

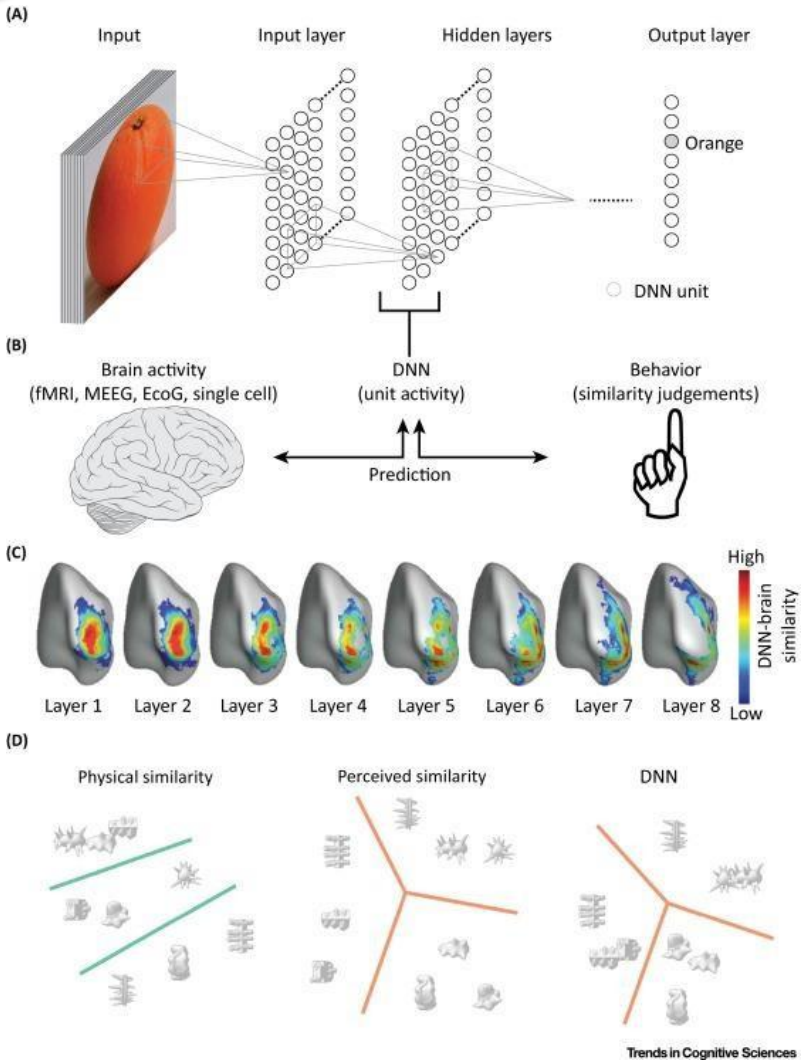


Wen, Haiguang, et al. "Neural encoding and decoding with deep learning for dynamic natural vision." *Cerebral cortex* 28.12 (2018): 4136-4160.



2

Artificial Neural Network



Cichy, Radoslaw M., and Daniel Kaiser. "Deep neural networks as scientific models." *Trends in cognitive sciences* 23.4 (2019): 305-317.

2

Break

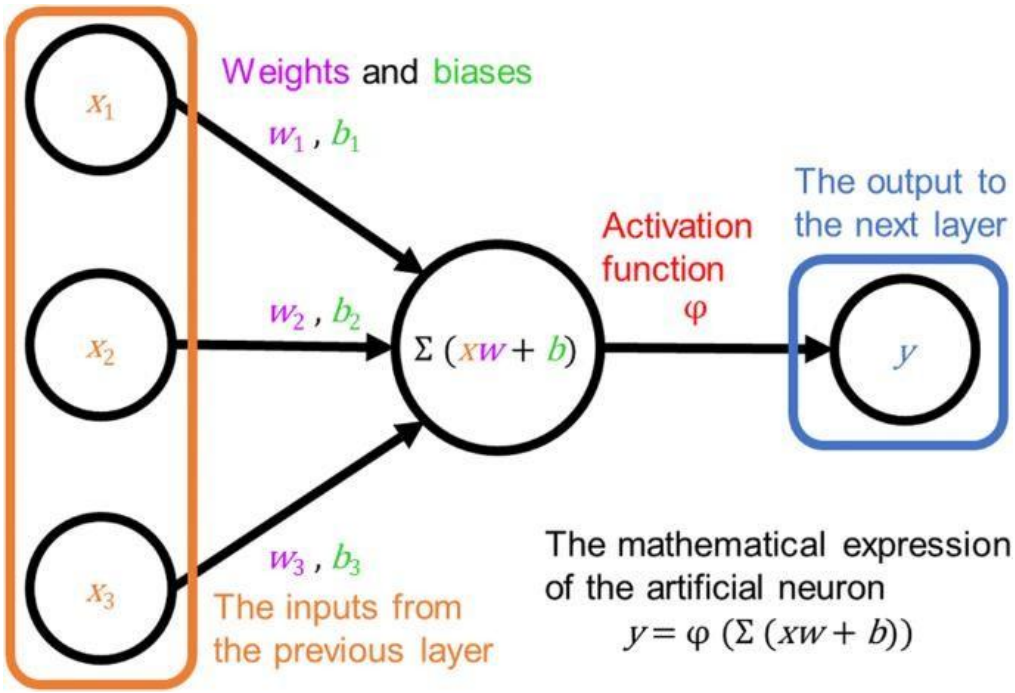
2

Artificial Neural Network

- From perceptron to neural network
  - Build a graph for nonlinear mapping
  - Revisit the perceptron

- Activation function of perceptron

$$h(x) = \begin{cases} 0 & (x \le 0) \\ 1 & (x > 0) \end{cases}$$ Step function

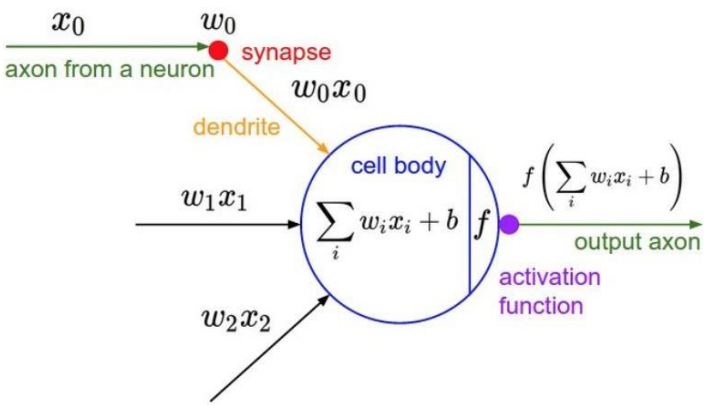


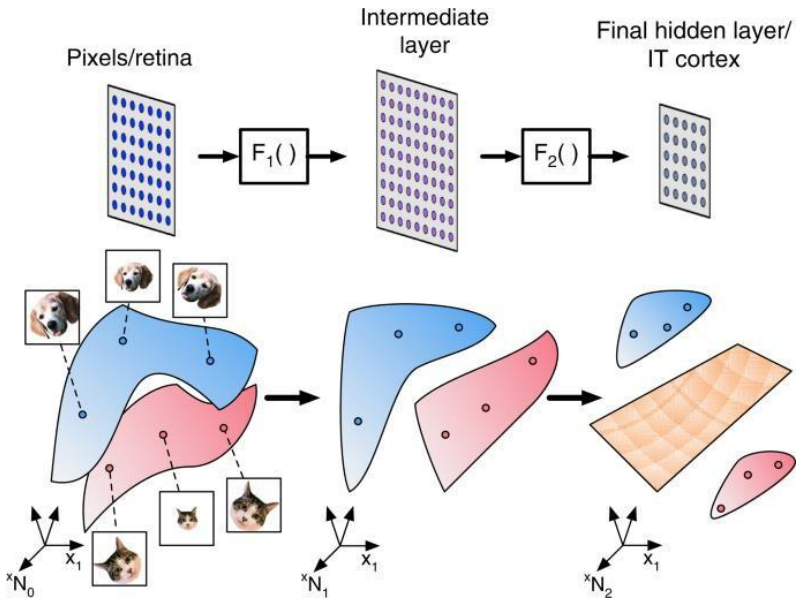
Wataya, Tomohiro, et al. "Introduction to deep learning: minimum essence required to launch a research." *Japanese journal of radiology* (2020): 1-15.

2

Artificial Neural Network

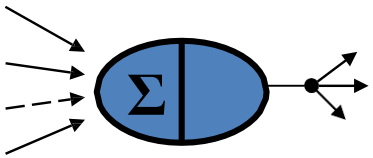
- Activation functions
  - “A node defines the output of that node given a set of inputs”
  - the end deciding what is to be fired to the next neuron
  - Non-linear conversion
  - Without activation function → linear model $W_2(W_1x) = Wx$

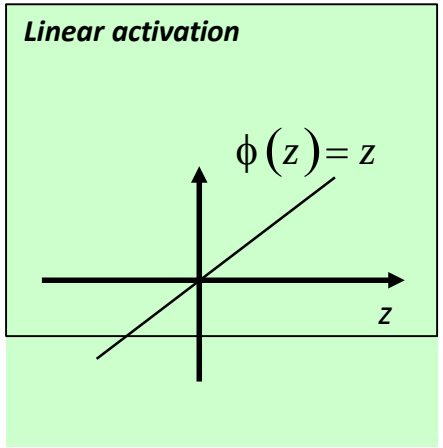


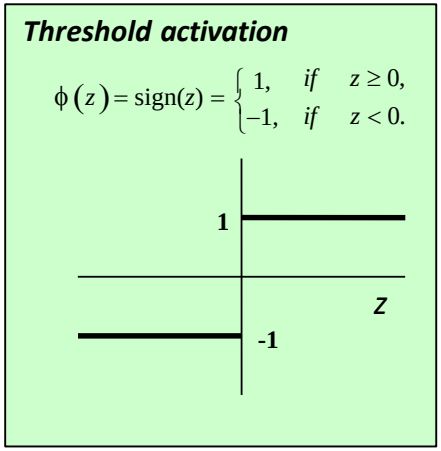


Cohen, Uri, et al. "Separability and geometry of object manifolds in deep neural networks." *Nature communications* 11.1 (2020): 1-13.

Activation Functions





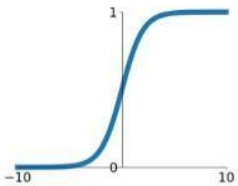


Artificial Neural Network

- Types of activation functions
  - Sigmoid activation family
  - Step function family
  - ReLU function family

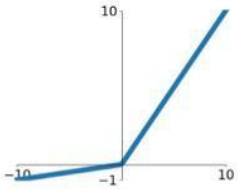
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



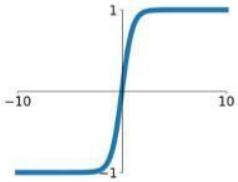
Leaky ReLU

$\max(0.1x, x)$



tanh

$\tanh(x)$

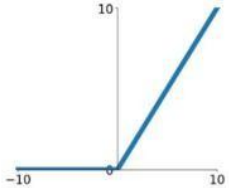


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

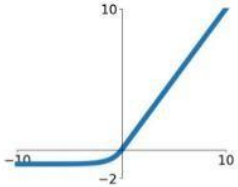
ReLU

$\max(0, x)$



ELU

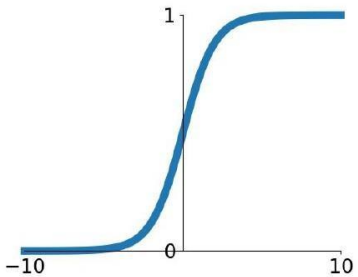
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Kim, Daeho, Jinah Kim, and Jaeil Kim. "Elastic exponential linear units for convolutional neural networks." *Neurocomputing* 406 (2020): 253-266.

Activation functions

Activation Functions



Sigmoid

$$\sigma(x) = 1/(1 + e^{-x})$$

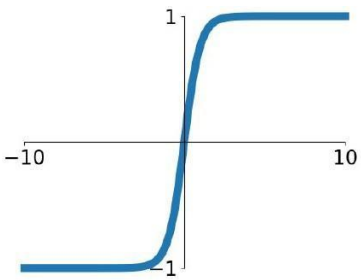
- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

3 problems:

1. Saturated neurons “kill” the gradients
2. Sigmoid outputs are not zero-centered
3. exp() is a bit compute expensive

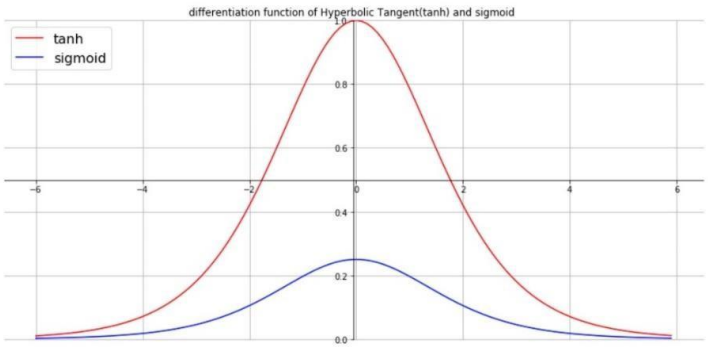
Activation functions

- **Tanh**



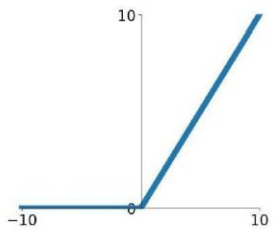
tanh(x)

- Squashes numbers to range [-1,1]
- zero centered (nice)
- still kills gradients when saturated :(

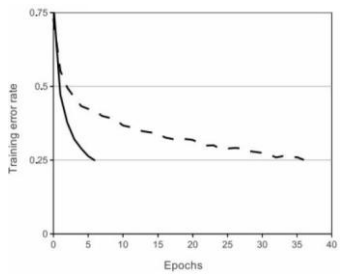


Activation functions

- **Rectified Linear Unit**



ReLU
(Rectified Linear Unit)



- Computes **f(x) = max(0,x)**

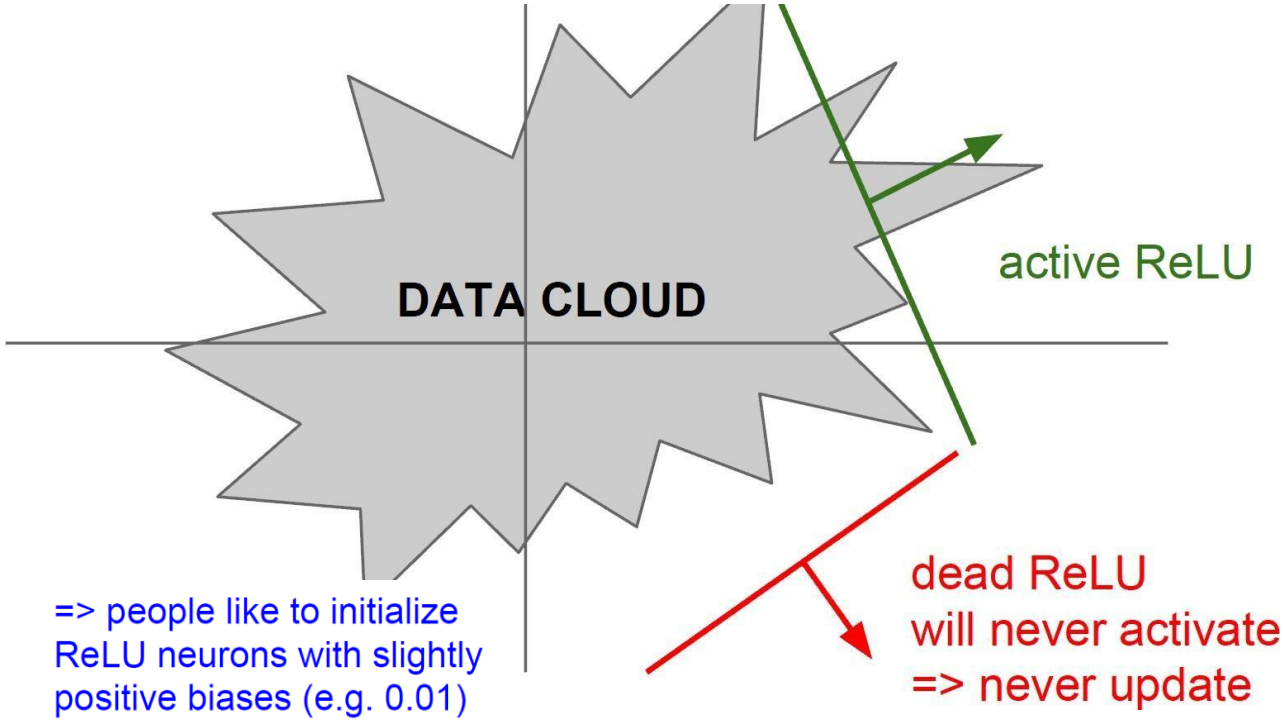
- Does not saturate (in +region)
- Very computationally efficient
- Converges much faster than sigmoid/tanh in practice (e.g. 6x)
- Actually more biologically plausible than sigmoid

- Not zero-centered output
- An annoyance:

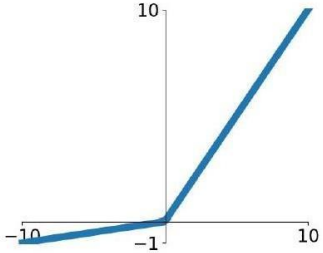
hint: what is the gradient when x < 0?

Activation functions

- **Rectified Linear Unit**



Activation functions



Leaky ReLU

$$f(x) = \max(0.01x, x)$$

some small value

- Does not saturate
- Computationally efficient
- Converges much faster than sigmoid/tanh in practice! (e.g. 6x)
- **will not "die".**

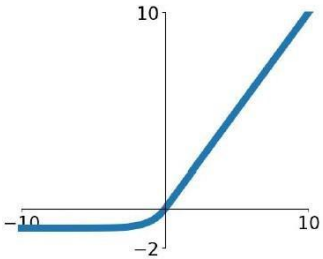
Parametric Rectifier (PReLU)

$$f(x) = \max(\alpha x, x)$$

backprop into α (parameter)

Activation functions

Exponential Linear Units (ELU)



- All benefits of ReLU
- Closer to zero mean outputs
- Negative saturation regime compared with Leaky ReLU adds some robustness to noise

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

- Computation requires exp()

Activation functions

- **MaxOut**
  - Does not have the basic form of dot product -> nonlinearity
  - Generalizes ReLU and Leaky ReLU
  - Linear Regime! Does not saturate! Does not die!

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

Problem: doubles the number of parameters :(

Activation functions

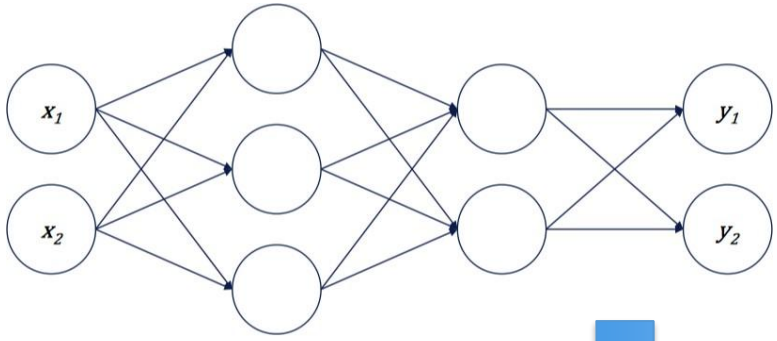
TLDR: In practice:

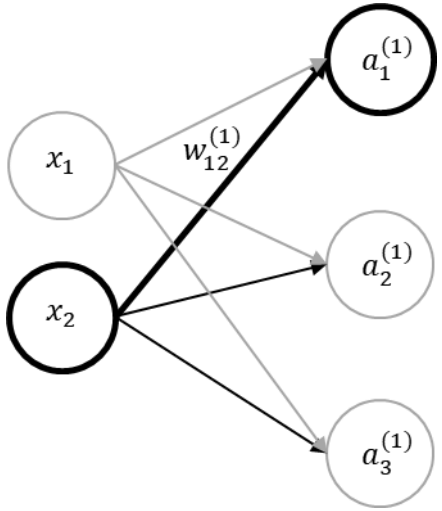
- Use ReLU. Be careful with your learning rates
- Try out Leaky ReLU / Maxout
- Try out tanh but don’t expect much
- Never use sigmoid

3

DL from Scratch

- Using numpy
- Tutorial_2_1_Introduction.ipynb
- ANN with two hidden layers
  - Activation function: sigmoid function





$w_{12}^{(1)}$

1st hidden layer

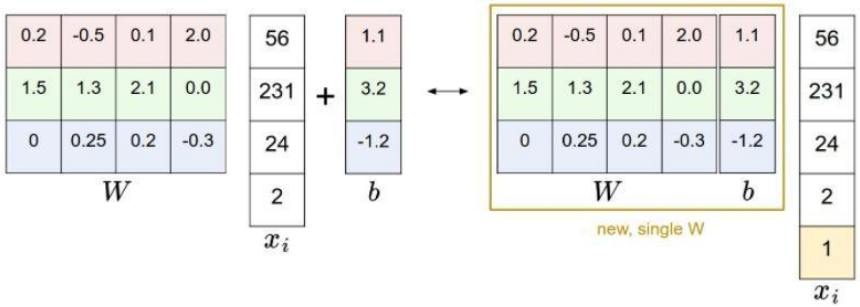
Previous node idx

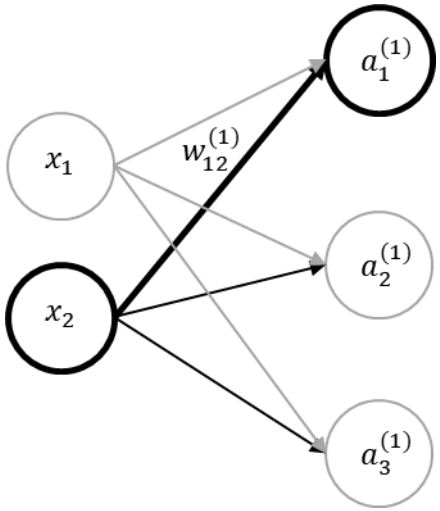
Current node idx

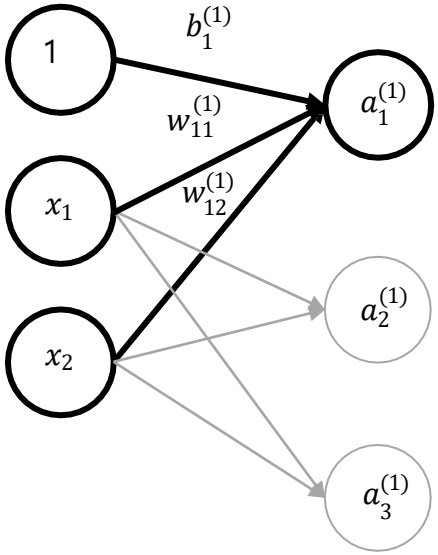
DL from Scratch

- Add bias at input layer
  - Bias trick
    - ✓ Extend input vector with one additional dimension
    - ✓ Simple matrix-vector multiplication

https://cs231n.github.io/linear-classify/







$$\boldsymbol{A}^{(1)} = \boldsymbol{W}^{(1)}\boldsymbol{x}$$

3

DL from Scratch

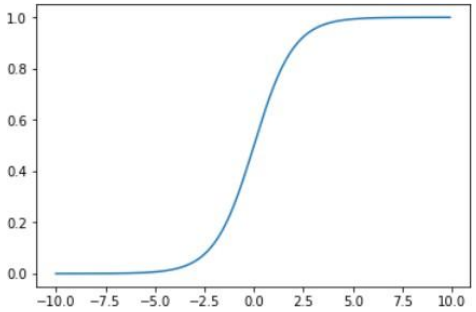
- Input to 1st hidden layer

$$\boldsymbol{A}^{(1)} = \boldsymbol{W}^{(1)}\boldsymbol{x} + \boldsymbol{b}$$

$$\boldsymbol{H}^{(1)} = \boldsymbol{sigmoid}(\boldsymbol{A}^{(1)})$$

```python
# input
x = np.array([1.0, 0.5])

# weight to map input(dim = 2) to the first hidden features (dim = 3)
# 2 x 3 matrix
# fully-connected layer
weight1 = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]])

# bias for the first hidden features (3 x 1)
bias1 = np.array([0.1, 0.0, 0.2])

# first hidden feature
a1 = np.dot(x, weight1) + bias1

# activation function
h1 = sigmoid(a1)

print(x.shape, h1.shape, a1.shape)
print(h1)
plt.figure()
plt.plot(a1, h1)
plt.show()
```

```python
import numpy as np
import matplotlib.pylab as plt

def sigmoid(h):
    return 1 / (1+np.exp(-h))

# test visualization of sigmoid
x = np.arange(-10, 10, 0.1)
y = sigmoid(x)
plt.figure()
plt.plot(x, y)
plt.show()
```

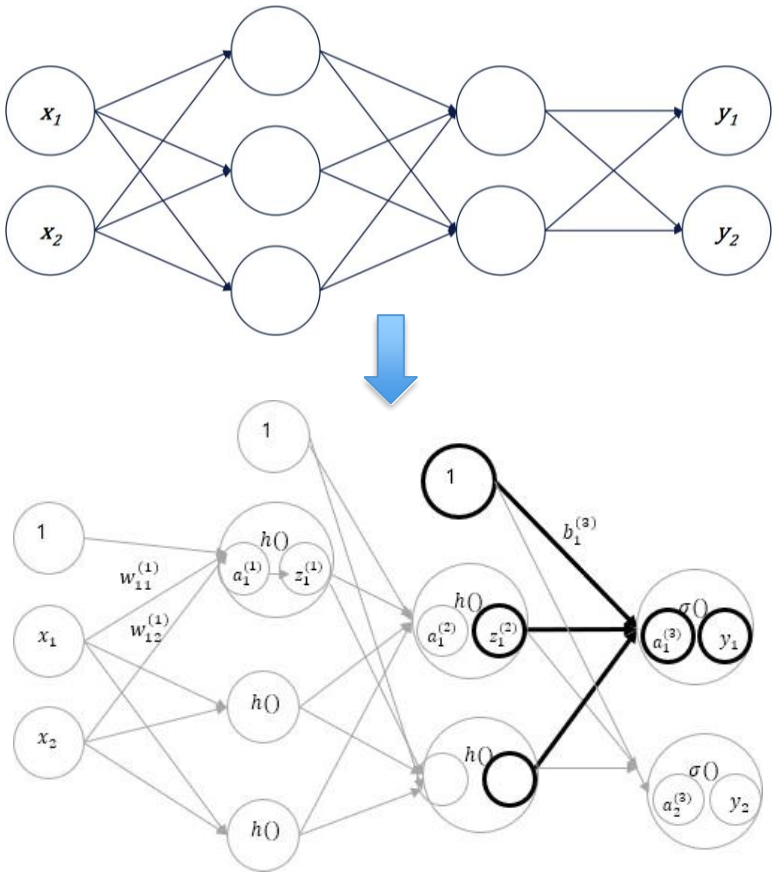


3

DL from Scratch

- Feedforward network

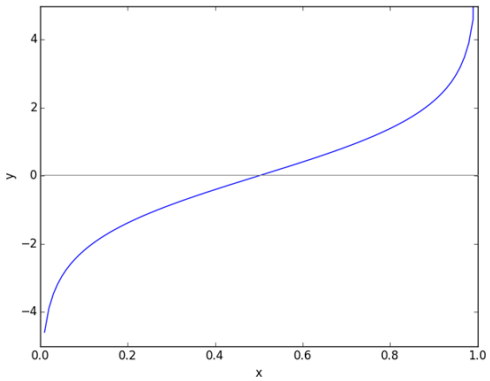
```python
import numpy as np

def sigmoid(h):
    return 1 / (1+np.exp(-h))

class ANN:
    def __init__(self):
        # weight to map input(dim = 2) to the first hidden features (dim = 3)
        self.weight1 = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]])
        self.bias1 = np.array([0.1, 0.0, 0.2])

        # first hidden layer --> second hidden layer (dim 3 --> dim 2)
        self.weight2 = np.array([[0.1, 0.3], [0.2, 0.4], [0.1, 0.7]])
        self.bias2 = np.array([0.1, 0.3])

        # second hidden layer --> output layer (dim 2 --> dim 2)
        self.weight3 = np.array([[0.1, 0.3], [0.5, 0.7]])
        self.bias3 = np.array([0.1, 0.1])

    def forward(self, x):
        # input --> 1st hidden layer
        a1 = np.dot(x, self.weight1) + self.bias1
        h1 = sigmoid(a1)

        # 1st --> 2nd
        a2 = np.dot(h1, self.weight2) + self.bias2
        h2 = sigmoid(a2)

        # 2nd --> output
        a3 = np.dot(h2, self.weight3) + self.bias3
        out = sigmoid(a3)

        return out
```



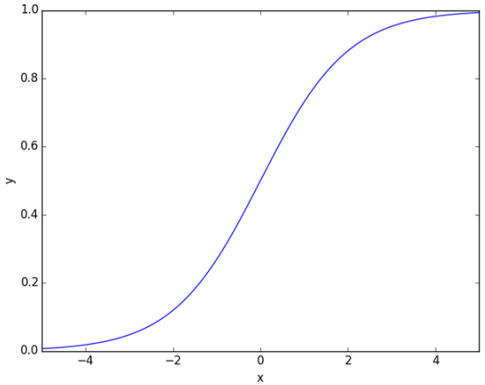

```python
net = ANN()
x = np.array([1.0, 5.0])
y = net.forward(x)
print(y)
```

DL from Scratch

- Activation function for output layer
  - **Tasks: classification or regression**
- Logistic regression
  - A process of modeling the probability of a discrete outcome given input
  - Input → categorical output



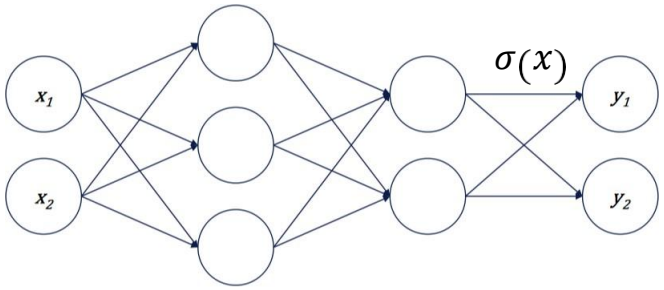
- **Sigmoid**
  - Odds: $\frac{p}{1-p}$, where p is a probability of an event
  - Logit function: logarithm of the odds
    - ✓ $logit(x) = \log(\frac{x}{1-x})$
    - ✓ Maps probabilities to the full range of real numbers
  - Sigmoid function: inverse function of the logit function
    - ✓ $\sigma(x) = \frac{1}{(1+\exp(-x))}$
    - ✓ Input: logit score, output: probability



3

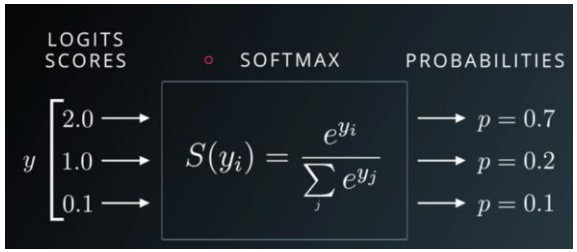
DL from Scratch

- Sigmoid function



Probability for individual event

- Softmax function
  - Normalize inputs into a probability distribution proportional to their exponentials
  - $softmax(h_j) = \frac{\exp(h_j)}{\sigma_{k=1}^{K} \exp(h_k)} \; for\ j = 1, \dots, K$
  - Sum of softmax function outputs = 1
  - Multi-class classification (K classes)



Udacity Deep Learning Slide on Softmax

DL from Scratch

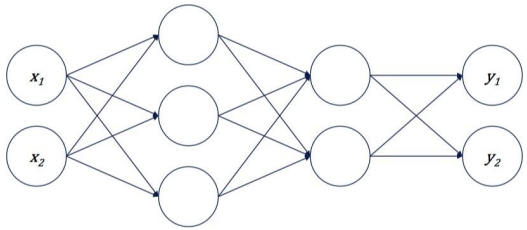
- Softmax function

```python
def softmax(h):
    exp_out = np.exp(h)
    sum = np.sum(exp_out)
    return exp_out/sum

class ANN:
    def __init__(self):
        # weight to map input(dim = 2) to the first hidden features (dim = 3)
        self.weight1 = np.array([[0.1, 0.3, 0.5], [0.2, 0.4, 0.6]])
        self.bias1 = np.array([0.1, 0.0, 0.2])

        # first hidden layer --> second hidden layer (dim 3 --> dim 2)
        self.weight2 = np.array([[0.1, 0.3], [0.2, 0.4], [0.1, 0.7]])
        self.bias2 = np.array([0.1, 0.3])

        # second hidden layer --> output layer (dim 2 --> dim 2)
        self.weight3 = np.array([[0.1, 0.3], [0.5, 0.7]])
        self.bias3 = np.array([0.1, 0.1])

    def forward(self, x):
        # input --> 1st hidden layer
        a1 = np.dot(x, self.weight1) + self.bias1
        h1 = sigmoid(a1)

        # 1st --> 2nd
        a2 = np.dot(h1, self.weight2) + self.bias2
        h2 = sigmoid(a2)

        # 2nd --> output
        a3 = np.dot(h2, self.weight3) + self.bias3
        # out = sigmoid(a3)
        out = softmax(a3)

        return out
```

- To avoid overflow problem

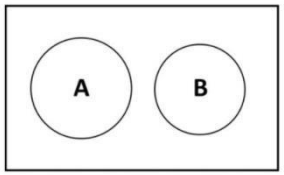
```python
def softmax_modified(h):
    max_logit = np.max(h)
    dif_h = h - max_logit
    exp_out = np.exp(dif_h)
    sum = np.sum(exp_out)
    return exp_out/sum
```

- Output: mutually exclusive



$$P(A \text{ or } B) = P(A) + P(B)$$

$$P(A \text{ and } B) = 0$$

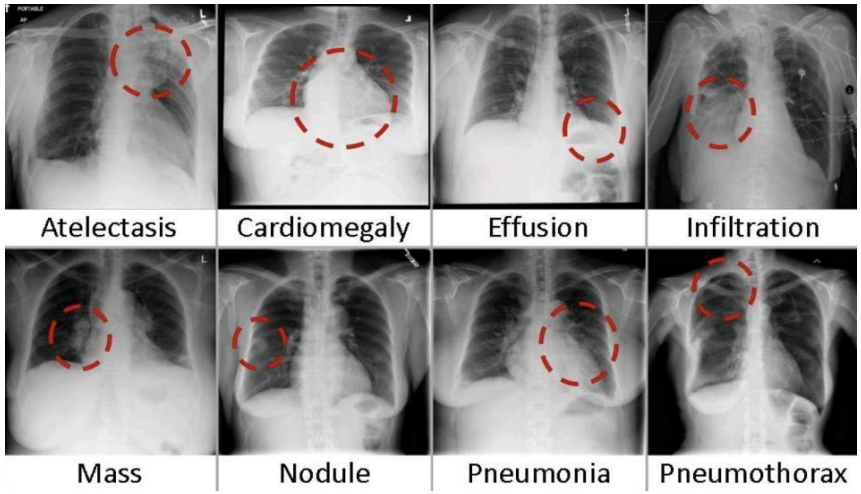


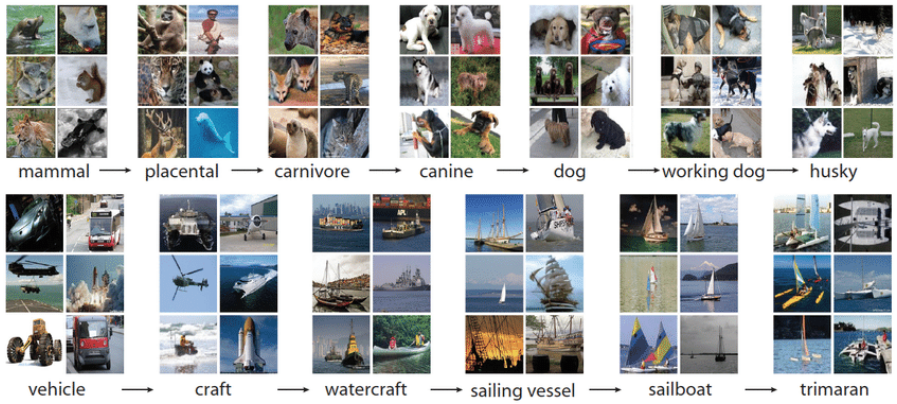
Note: No overlap between A and B

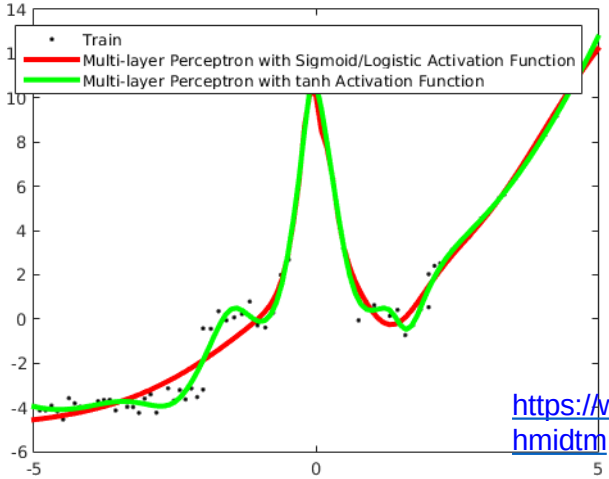
3

DL from Scratch

- Softmax / Sigmoid / …





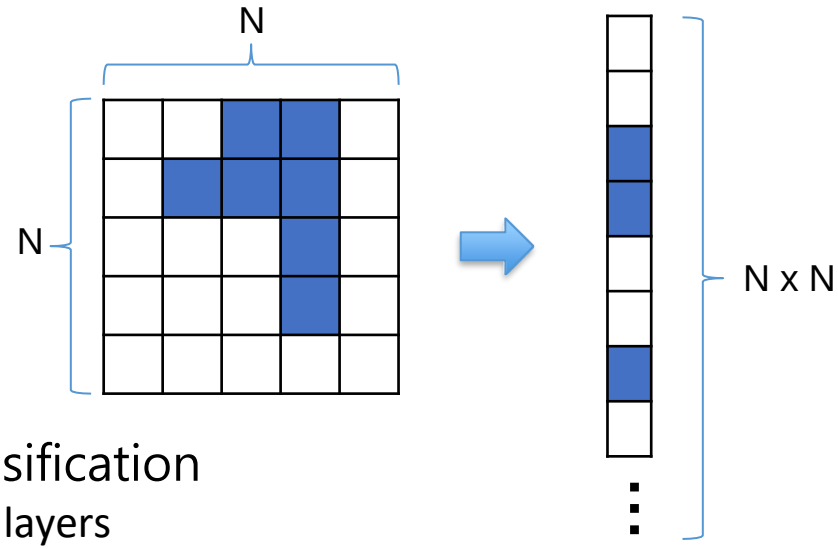


https://www.cs.ubc.ca/~schmidtm

3

DL from Scratch

- MNIST
  - Dataset of handwritten digits
  - http://yann.lecun.com/exdb/mnist/
  - Train: 60,000, test: 10,000
  - Image: 28 x 28
  - Label: 10 classes



- Build your network for MNIST classification
  - Multi-layer perceptron with 3 hidden layers
  - Logistic regression

	Input	Hidden 1	Hidden 2	Hidden 3	Output
Feature	$28 \times 28 = 784$ (Flatten)				10×1
Weights	784×128				
Bias	128				
Activation	Sigmoid				

3

DL from Scratch

- How to learn parameters from data
  - Parameters: weights, bias

- Computational Graph
- Backpropagation
- Optimization
  - Gradient descent
  - Problems
  - Optimizers
- Losses for tasks
- Training/validation/testing

A Ω
Our Family UCU Our Pride